

IPP Půlsemestrálky – Otázky a odpovědi (všechny roky)

2025

(A)

1) Co obsahuje zpráva a popsat veřejný protokol v třídním OOJ.

Zpráva se skládá z:

- **Selektor** (jméno/identifikátor zprávy)
- **Odkaz na příjemce** (objekt, kterému je zpráva zaslána)
- **Argumenty/parametry** (další objekty upřesňující význam)

Odesílatel **NENÍ** součástí zprávy – je uložen v zásobníku kontextů. Návratová hodnota metody se nevrací formou další zprávy – předání výsledku zajišťuje objektový model.

Veřejný protokol (objekt přijímá zprávu od jiného objektu):

- a) **Invokace metody** – příjemce nalezne odpovídající metodu (dle selektoru) ve své třídě, pokud nenalezena, hledá se v nadtřídách (procházení hierarchie dědičnosti). Po nalezení se metoda invokuje a výsledek se vrátí odesílateli.
- b) **Chyba** – metoda nenalezena nikde v hierarchii → výjimka `DoesNotUnderstand`.

2) Problém vícenásobné třídní dědičnosti a jak se řeší.

Problém: Podtřída dědí od dvou+ nadtříd, které mohou mít položky (metody/atributy) stejného jména a typu. Při invokaci na instanci podtřídy není jednoznačné, kterou implementaci zvolit.

Řešení kolize metod stejného jména a typu:

1. **Zakázat** – sémantická kontrola při kompilaci ohlásí chybu
2. **Skrytí** – jedna z kolizních metod přístupná jen v kontextu své nadtřídy
3. **První nalezený výskyt** – dle pořadí zápisu nadtříd a prohledávání do hloubky (Python MRO)
4. **Povinná redefinice** – podtřída musí kolizní metodu reimplementovat
5. **Plná kvalifikace** – při volání specifikace nadtřídy: `A::m()` (C++)

Kolize atributů: zakázat, skryt, sloučit (Python – jeden výskyt, vazba přes jméno), replikace s konzistencí. Atributy stejného jména ale různých typů jsou vždy zakázány.

Další problémy: pořadí volání konstruktorů (DFS dle zápisu nadtříd), uložení v paměti (diamantový problém).

3) Deklarace vs definice.

- **Deklarace** – vymezuje atributy entity (jméno, typ, signatura). Nealokuje paměť, neobsahuje tělo funkce. Může být explicitní i implicitní.
- Proměnná: `extern int x;`
- Funkce: `int f(int, double);`
- **Definice** – vymezuje atributy entity a navíc: u proměnné zahrnuje **alokaci paměti** (příp. inicializaci), u funkce navíc **tělo** (implementaci).
- Proměnná: `int x = 54;`

- Funkce: `int f(int i, double d) { return i + (int)d; }`

V kontextu OOP: **rozhraní** obsahuje pouze deklarace metod, **třída** obsahuje definice metod.

4) Výpočet adresy $a[x]$, pole $[13..24]$, $base=64$, $int=4B$.

$$adresa(a[x]) = 64 + (x - 13) \times 4$$

Příklad: $a[15] = 64 + (15 - 13) \times 4 = 64 + 8 = 72$

(B)

1) Self v instanční a třídní metodě + protokol + technické předání.

- **V instanční metodě:** self-parametr = **instance** třídy → může přistupovat k hodnotám instančních atributů příjemce zprávy
- **V třídní metodě:** self-parametr = **objekt třídy** → nemůže přistupovat k instančním atributům, protože neví, ze které instance je brát (žádná instance nemusí existovat)
- Používá **interní protokol** (zpráva sám sobě): a) přístup k atributu (čtení/modifikace) b) invokace metody c) chyba DoesNotUnderstand
- Technicky: self je implementován jako **skrytý (nultý) parametr** metody, předáván implicitně jako odkaz na příjemce zprávy
- Self **NENÍ uložen v instanci ani třídě** – je to jen proměnná dostupná uvnitř invokované metody
- Self je aktuální i při volání zděděné metody – odkazuje na skutečného příjemce

2) Objekt v beztřídním OOJ + delegace + proč se používá.

Objekt v SELF má unifikovanou strukturu – seznam **slotů**:

- **Datový slot** – obsahuje hodnotu/referenci na jiný objekt
- **Metodový slot** – obsahuje kód metody
- **Rodičovský slot** (parent*) – odkaz na rodičovský objekt pro delegaci

Nerozlišuje se mezi instancemi a třídami – všechny objekty mají stejnou strukturu.

Delegace: objekt nemá odpovídající slot → přepošle zprávu přes *parent slot na rodičovský objekt*.

Klíčový rozdíl od prostého předání zprávy: self zůstává původní příjemce (nemění se!).* Metoda nalezená v rodičovském objektu tak pracuje v kontextu původního příjemce.

Proč se používá: realizuje dědičnost v beztřídních OOJ. Sdílené chování (metody) se umísťuje do rysů (traits), objekty na ně delegují. Více parent* slotů = vícenásobná dědičnost. Měnitelné za běhu = dynamická dědičnost.

3) Sémantika + 2 věci vyhodnocované za běhu v C.

Sémantika = popis/definice významu jednotlivých syntaktických konstrukcí, způsobu jejich vyhodnocení a zpracování. Dělí se na statickou (při překladu) a dynamickou (za běhu).

2 věci vyhodnocované za běhu v C (dynamická sémantika):

1. **Kontrola indexu pole** – pokud je index dán výrazem, nelze ověřit při překladu, zda nepřekročí hranice
2. **Dereference ukazatele** – zda ukazatel neukazuje na neplatnou/uvolněnou paměť (NULL, dangling pointer)

4) C struct v paměti + výhody/nevýhody.

Položky struktury se ukládají **za sebou v pořadí deklarace** jako souvislý blok paměti. Překladač může vkládat **výplňové bajty (padding)** mezi položky kvůli zarovnání (alignment) – každá položka na adrese dělitelné její velikostí (požadavek architektury procesoru).

Přístup k položce = **bázová adresa struktury + offset** dané položky. Rozdíl oproti přístupu k proměnné: k proměnné přistupujeme přímo přes její adresu, u struktury je navíc krok výpočtu offsetu.

Výhody: logické seskupení souvisejících dat, efektivní přístup přes staticky známý offset, heterogenní data v jednom celku. **Nevýhody:** padding může plýtvat pamětí, pořadí položek ovlivňuje velikost, nelze dynamicky měnit strukturu.

(C)

1) Rozhraní + abstraktní třída + simulace rozhraní.

Rozhraní (interface) je schéma deklarující:

- Seznam **deklarací metod** (jména, parametry, návratové typy) – bez implementace
- Seznam **nadrozhraní** (rozhraní může dědit od více nadrozhraní)
- Neobsahuje definice metod ani instanční atributy

Třída implementující rozhraní musí implementovat všechny metody z rozhraní **i ze všech jeho nadrozhraní**. Význam: v OOO s jednoduchou dědičností umožňuje třídě implementovat více rozhraní → sdílení protokolu bez problémů s kolizí implementací.

Abstraktní třída – obsahuje min. 1 abstraktní metodu (bez implementace), nelze instanciovat. Na rozdíl od rozhraní může mít i implementované metody a atributy (částečná implementace).

Simulace rozhraní: V OOO s vícenásobnou dědičností → abstraktní třída se **všemi** metodami abstraktními + **bez** atributů. Podmínka: jazyk musí podporovat vícenásobnou dědičnost takových abstraktních tříd.

2) Aktivační objekt v SELF.

Vzniká při **invokaci metody** – objekt metody se naklonuje → vzniká aktivační objekt, který tvoří jmenný prostor (kontext) dané invokace.

Sloty aktivačního objektu:

- **Self/receiver** (implicitní) – odkaz na původního příjemce zprávy; v SELFu klíčové slovo `self` chybí, je implicitní
- **Argumenty/parametry** – hodnoty předané při zaslání zprávy
- **Lokální proměnné** – proměnné definované uvnitř metody
- **Parent*** – odkaz na objekt, kde byla metoda definována (lexikální kontext)

3) Syntaxe + výrazný rys Pythonu.

Syntaxe = definice struktury programu, tj. jakým způsobem je dovoleno jednotlivé konstrukce řadit za sebe. Dnes se popisuje: (E)BNF, syntaktické grafy/diagramy, bezkontextové gramatiky, slovní popis s příklady.

Výrazný rys Pythonu: Bílé znaky (odsazení/indentace) jsou **součástí syntaxe** – definují bloky kódu místo `{ }` nebo `begin/end`. Nesprávné odsazení je syntaktická chyba.

4) Vlastnosti proměnné + co ovlivnit za běhu v PHP.

6 vlastností proměnné: jméno, adresa/umístění v paměti, typ, hodnota, doba života, rozsah platnosti.

Za běhu v PHP lze ovlivnit:

- **Typ** – přiřazením hodnoty jiného typu, PHP je dynamicky typovaný (`$x = 5; $x = "ahoj";`)
- **Hodnotu** – přiřazením nové hodnoty
- **Dobu života** – voláním `unset($x)` pro explicitní zrušení; deklarací `static` v rámci funkce pro prodloužení
- **Rozsah platnosti** – klíčové slovo `global` ve funkci zpřístupní globální proměnnou v lokálním rozsahu

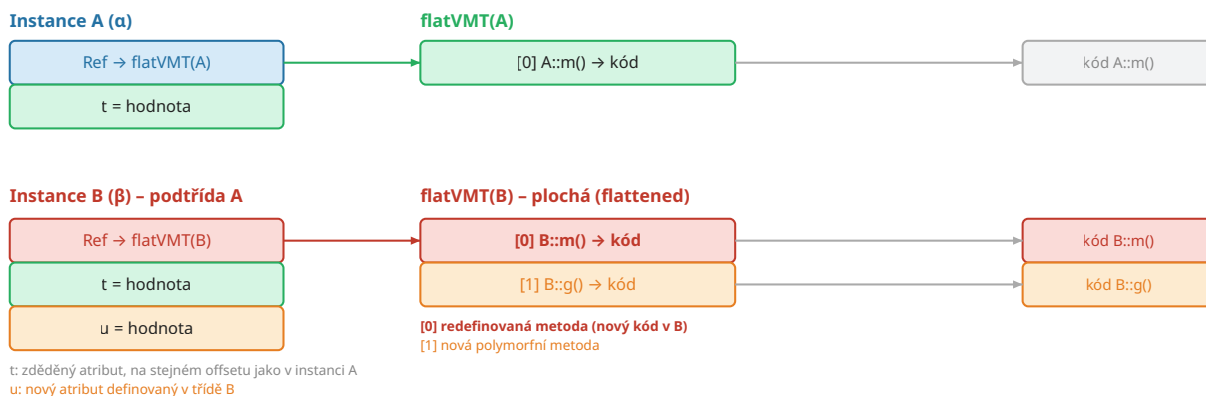
Nelze ovlivnit: jméno, adresu/umístění (řeší interpret).

(F)

1) VMT – nakreslit uložení instance v paměti.

Uložení instance v paměti – jednoduchá dědičnost (optimalizovaná plochá VMT)

Třída A: instanční atribut `t`, polymorfní metoda `m()` · Třída B extends A: atribut `u`, polymorfní `g()`, redefinuje `m()`



Důležité:

- Nepolymorfní a statické metody se do VMT NEUKLÁDAJÍ (brzká vazba).
- Zděděné atributy v B mají STEJNÉ offsety jako v A (zahrnutí/subsumption).
- Invokace: instance → Ref(VMT) → offset v tabulce → kód metody.
- `super.m()` = brzká vazba (staticky známo kde hledat).

Instance obsahuje: odkaz na plochou VMT (flatVMT) své třídy + hodnoty instančních atributů (včetně zděděných – ty na stejných offsetech jako u nadtřídy).

flatVMT obsahuje ukazatele na kód **pouze polymorfních (virtuálních)** metod. Nepolymorfní a statické metody se do VMT NEUKLÁDAJÍ – volají se přes brzkou (statickou) vazbu. U podtřídy: flatVMT kopíruje strukturu VMT nadtřídy + redefinované mají nové ukazatele + nové polym. metody na konci.

Invokace: instance → načti Ref(VMT) → přístup na offset metody v tabulce → zavolej kód.

2) Implicitní rušení objektu + 2 nevýhody + kdy se může zrušit.

Garbage Collector (GC) automaticky uvolní objekt, když na něj neexistuje žádný programátorovi přístupný odkaz. GC dokáže uvolnit i **shluky** objektů (navzájem se odkazují, ale zvenčí nedosažitelné). Uvolnění **nenastane ihned** – provádí se v dávkách, někdy po splnění podmínky nedosažitelnosti. Před uvolněním může být invokována finalizační metoda (vykonání není garantováno, čas nepředvídatelný).

2 nevýhody:

1. **Nedeterminismus** – programátor nemůže předvídat, kdy se GC spustí; může způsobit zpomalení v nevhodný moment

2. **Režie** – prostorová (metadata o stavu objektů) i časová (samotné provádění GC, udržování počtů referencí)

3) Statické typování + proč deklarace typu + operace + v C.

Staticky typovaný jazyk = typ proměnné je určen před během programu (při překladu). Většina typových kontrol při kompilaci.

Proč se zavedla deklarace typu: umožňuje typovou kontrolu při překladu (odhalení chyb dříve), překladač může určit velikost paměti pro alokaci, efektivnější generování kódu.

Operace + na primitivních typech v C (operátor je přetížený – sémantika závisí na typech):

- `int + int → int`
- `float + float → float`
- `int + float → float` (implicitní konverze `int → float`, koerce)
- `pointer + int → pointer` (pointerová aritmetika – posun o násobek velikosti typu)
- `char + char → int` (char se chápe jako malé celé číslo)

4) C pointer – co obsahuje, co umožňuje, co podporuje.

Obsahuje: adresu v paměti + typ odkazovaného objektu (překladač ví, jak interpretovat data a jakou velikost mají).

Co umožňuje (bez pointerů by nešlo):

- Dynamická alokace paměti na haldě (malloc/free)
- Nepřímý přístup k datům (přes adresu místo jména)
- Předávání odkazem (funkce může modifikovat proměnnou volajícího)
- Dynamické datové struktury (spojové seznamy, stromy, grafy)

Co podporuje: pointerová aritmetika (`p + n`), dereference (`*p`), vícenásobná indirection (`**pp`), funkční ukazatele (callbacks).

Rizika: visící ukazatel (dangling pointer), úbytek paměti (memory leak), NULL dereference, přetečení bufferu.

2024

(A)

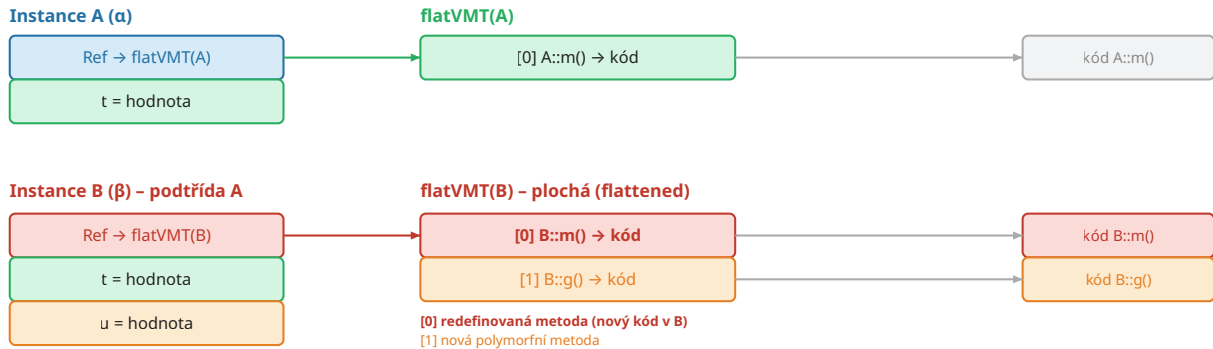
1) Instanciace – čistý vs hybridní OOJ.

- **Čistý třídní OOJ** (Smalltalk): zaslání **třídní zprávy** objektu třídy – třída je objekt první kategorie. Např. `t := Time now.` nebo `p := Person new.`
- **Hybridní třídní OOJ** (Java, C++, PHP): klíčové slovo **new** + jméno třídy + parametry konstrukturu. Např. `Car c = new Car();` nebo `$p = new ShopProduct();`

2) Uložení instance A a B s VMT (A: atribut t, polym. m; B extends A: atribut u, polym. g).

Uložení instance v paměti – jednoduchá dědičnost (optimalizovaná plochá VMT)

Třída A: instanční atribut t, polymorfní metoda m(). Třída B extends A: atribut u, polymorfní g(), redefinuje m()



t: zděděný atribut, na stejném offsetu jako v instanci A
u: nový atribut definovaný v třídě B

Důležité:

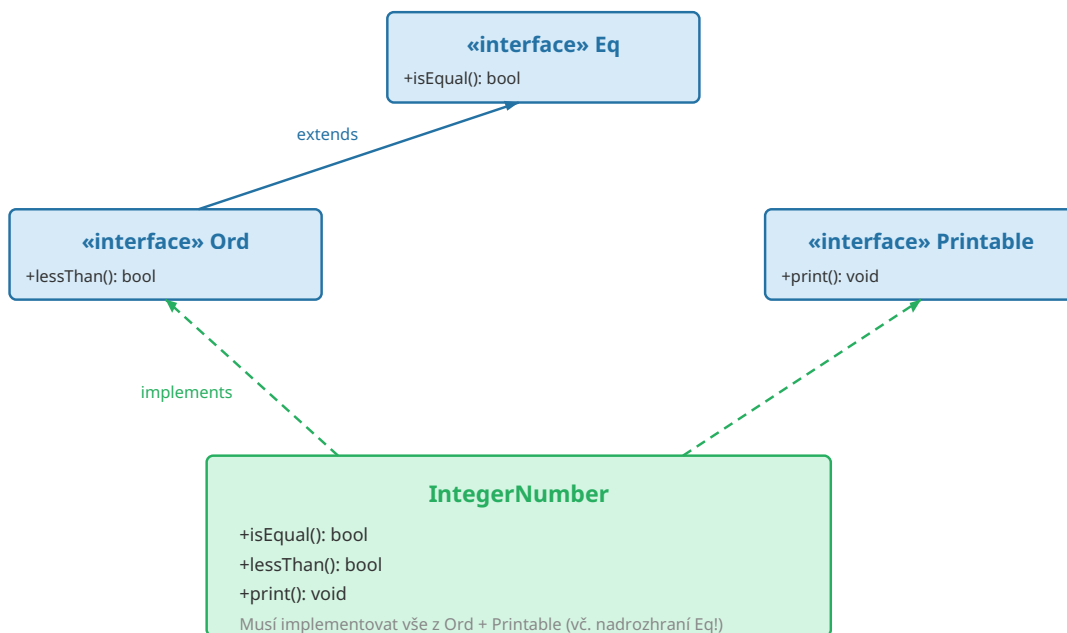
- Nepolymorfní a statické metody se do VMT NEUKLÁDÁJÍ (brzká vazba).
- Zděděné atributy v B mají STEJNÉ offsety jako v A (zahrnutí/subsumption).
- Invokace: instance → Ref(VMT) → offset v tabulce → kód metody.
- super.m() = brzká vazba (staticky známo kde hledat).

Instance A: Ref(flatVMT(A)) + hodnota t. **flatVMT(A):** [0] → A::m(). **Instance B:** Ref(flatVMT(B)) + hodnota t (zděděný, stejný offset) + hodnota u (nový). **flatVMT(B):** [0] → A::m() (zděděná) + [1] → B::g() (nová).

Plochá VMT = všechny polym. metody (i zděděné) v jedné tabulce → méně dereferencí, stejné offsety. Nepolymorfní a statické metody NEJSOU ve VMT (brzká vazba).

3) Rozhraní + abstraktní třída + simulace.

Rozhraní (Interface) – koncept



Viz 2025 C1 pro plnou odpověď.

4) Sémantika + 2 typy kontroly při překladu v C.

Sémantika = popis/definice významu syntaktických konstrukcí, způsobu vyhodnocení a zpracování.

Dva typy kontroly při překladu v C: 1) **kontrola existence** proměnných/funkcí (zda deklarovány před použitím) 2) **typová kompatibilita** ve výrazech, parametrech funkcí, přiřazeních.

5) Adresa $a[x]$, pole [13..24], base=32, int=4B.

$\text{adresa}(a[x]) = 32 + (x - 13) \times 4$

(B)

1) Klonování v beztřídním OOJ.

Klonování = vytvoření nového objektu **mělkým kopírováním** (shallow copy) existujícího objektu (prototypu) – nový objekt má všechny sloty odkazující na totožné objekty jako předloha. Po klonování je třeba provést **modifikaci** – úprava hodnot datových slotů a potenciálně i strukturální změny. Jazyk: **SELF** (1986, dialekt Smalltalku).

2) Layout – viz 2024 A2.

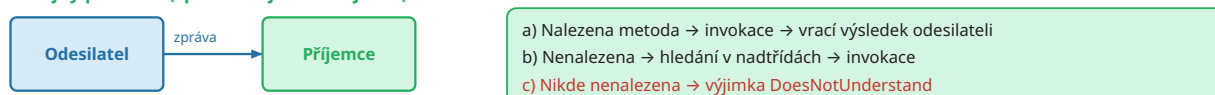
3) Zpráva + veřejný protokol v OOJ s jednoduchou dědičností.

Zpráva a protokol (veřejný + interní)

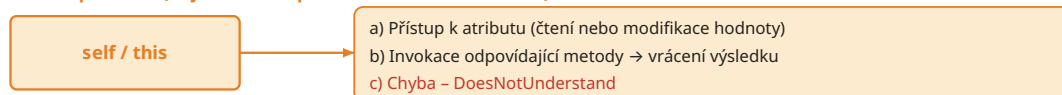
Zpráva (message) se skládá z:



Veřejný protokol (zpráva od jiného objektu):



Interní protokol (objekt zasílá zprávu sám sobě — self/this):



self/this = pseudoproměnná dostupná jen uvnitř metody. NENÍ uložena v instanci ani třídě!

Zpráva: selektor + odkaz na příjemce + argumenty. Odesílatel NENÍ součástí zprávy. **Veřejný protokol:**

a) Invokace metody (hledání ve třídě, pak nadtřídách) → výsledek odesílateli b) DoesNotUnderstand.

4) C struct v paměti – viz 2025 B4.

5) Vlastnosti proměnné + Python runtime.

6 vlastností: jméno, adresa/umístění, typ, hodnota, doba života, rozsah platnosti. V Pythonu za běhu: **typ** (přiřazením jiné hodnoty), **hodnotu** (přiřazením), **dobu života** (`del`), **rozsah platnosti** (`global` , `nonlocal`).

(C)

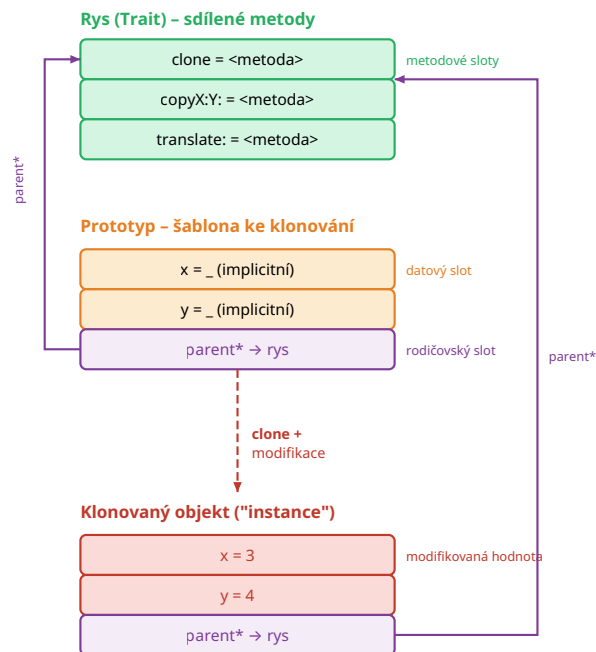
1) Explicitní rušení instance.

Programátor explicitně vyvolá rušení příkazem `delete` . Před uvolněním paměti se invokes **destruktor** – speciální metoda pro úklid (uvolnění zdrojů). Destruktoři nadtříd se volají v pořadí opačném konstruktorům. Po provedení destrukturu se objekt uvolní z paměti. Při chybě programátora → memory leaks. Příklad: C++.

2) Layout dvou tříd s dědičností – viz 2024 A2.

3) Delegace v SELF.

Beztrždní OOJ SELF – prototyp, rys, delegace



Delegace (klíčový mechanismus):

1. Objekt přijme zprávu
 2. Hledá odpovídající slot ve svých slotech
 3. Nenalezeno → delegace přes `parent*` slot
 4. **self = PŮVODNÍ příjemce (nemění se!)**
- Více `parent*` slotů = vícenásobná dědičnost

Dynamická dědičnost:

Hodnoty `parent*` slotů lze měnit za běhu
→ objekt dynamicky mění předka i chování

Shrnutí pojmů:

- Rys (trait) = objekt se sdílenými metodami (simuluje třídu)
- Prototyp = objekt s daty + `parent*` na rys (šablona)
- Delegace = přeposílání zprávy přes `parent*` (self se nemění!)

Viz 2025 B2 pro plnou odpověď. Klíčové: self = původní příjemce, nemění se.

4) Doba života + vztahy dvou proměnných v Pythonu.

Doba života = časový interval, po který je pro proměnnou alokována paměť.

Možné vzájemné vztahy dob života dvou proměnných: úplné překrytí, částečné překrytí, disjunkce (nikdy současně), totožná (začínají i končí současně), vnořená.

V Pythonu: globální = celý běh programu, lokální = doba provádění funkce, dynamické (objekty na haldě) = dokud existuje alespoň jedna reference (GC).

5) Netypovaný jazyk + * v Pythonu.

Netypovaný jazyk = proměnná nemá typ určen ve zdrojovém kódu, typ je dán aktuální hodnotou a může se za běhu měnit. Interpret zajišťuje implicitní konverze automaticky.

* v Pythonu na primitivní typy: `int*int→int`, `float*float→float`, `int*float→float` (konverze), `str*int→str` (opakování řetězce: `"ab"*3 = "ababab"`), `str*str→TypeError`.

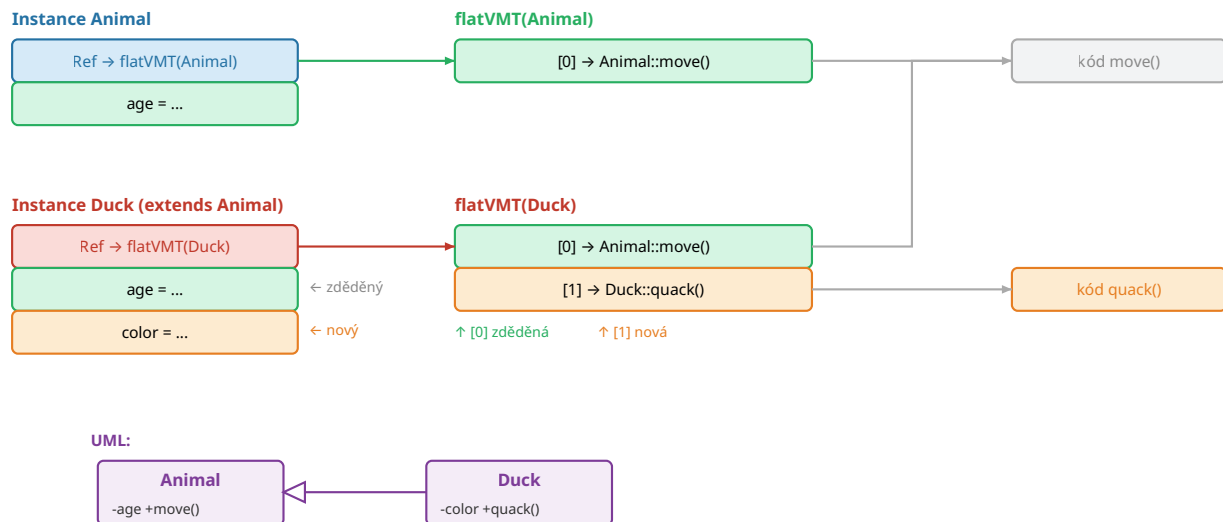
(D)

1) Konstruktor jako instanční vs třídní metoda.

- **Instanční metoda:** self-parametr = **již vytvořená instance** – konstruktor ji inicializuje. Příklad: `__init__` v Pythonu, konstruktory v Java/C++ (this = instance).
- **Třídní metoda:** self-parametr = **objekt třídy** – třída sama vytváří novou instanci. Příklad: `Smalltalk (Person new)` – třídní zpráva).

2) Animal/Duck layout.

Příklad: Animal / Duck – uložení instance v paměti



Instance Duck: `Ref(flatVMT(Duck)) + age` (zděděný, stejný offset) + `color` (nový). `flatVMT(Duck)`: `[0] → Animal::move()` (zděděná) + `[1] → Duck::quack()` (nová).

3) Kolize atributů u vícenásobné dědičnosti.

Řešení: 1) **Zakázat** 2) **Skrytí** (přístupný jen v kontextu nadtřídy) 3) **Sloučení** – jeden výskyt, vazba přes jméno (Python) 4) **Replikace** – dvakrát v paměti s udržováním konzistence. Atributy různých typů = vždy zakázáno.

4) Syntaxe + jak se popisuje + rys PHP.

Syntaxe = definice struktury programu. Popis: (E)BNF, syntaktické grafy, bezkontextové gramatiky, slovní popis.

PHP rysy: proměnné začínají `$`, míchání procedurální a OO syntaxe, `function / static` mají více významů dle kontextu.

5) Binární sčítání v netypovaném jazyce (2 typy podporující sčítání, 1 ne).

1. Zjistit typ levého a pravého operandu
2. Oba podporují sčítání, stejný typ → provést operaci, vrátit výsledek
3. Oba podporují, různé typy → provést konverzi jednoho na druhý → sčítání
4. Alespoň jeden nepodporuje → pokus o konverzi na podporující typ → pokud konverze neexistuje → chyba (`TypeError`)

(E)

1) Obě fáze instanciace.

1. **Alokace paměti** – přidělení paměti pro instanci (většinou na haldě). Vytvoření struktury s odkazem na třídu.
2. **Inicializace** – invokace konstruktoru (speciální metody pro nastavení atributů). Případné volání konstruktů nadtříd (nejprve nadtřídy, pak vlastní třída).

2) Layout – viz 2024 A2.

3) GC – kdy a jak uvolňuje paměť.

Rušení instancí – GC vs Explicitní

Explicitní (C++):

1. Programátor volá delete obj
2. Invokace destruktoru (~ClassName)
3. Destruktory nadříd (opačně než konstr.)
4. Uvolnění paměti

✓ Vykonání garantováno, čas předvídatelný
× Memory leaks při chybě programátora

Implicitní – GC (Java, Python):

1. Žádný přístupný odkaz na objekt
2. GC označí ke zrušení (i shluky!)
3. Někdy později: volání finalizační metody
4. Uvolnění paměti v dávkách

× Vykonání finalizace NENÍ garantováno
× Čas spuštění GC nepředvídatelný

Destruktor vs Finalizační metoda:

Destruktor: volaný programátorem (delete) · garantované vykonání · předvídatelný čas
Finalizace: volaná GC automaticky · vykonání NENÍ garantováno · čas nepředvídatelný

3 nevýhody GC:

- 1) Prostorová režie (metadata o stavu objektů)
- 2) Časová režie (udržování informací + samotné provádění GC)
- 3) Nedeterminismus (kdy se GC spustí? může způsobit zpomalení v nevhodný moment)

Viz 2025 F2. GC uvolní objekt bez přístupného odkazu (i shluky). Ne ihned – v dávkách. Finalizační metoda negarantována.

4) Union v C.

Variantní záznam ukládá všechny položky **přes sebe** (na stejné adrese) – ne za sebou! Velikost = velikost **největší** položky. Použití: úspora paměti (jen jedna aktivní položka v daný moment), reinterpretace bitového vzoru hodnoty.

5) Rozsah platnosti + Python.

Rozsah platnosti = ta část programu, kde s proměnnou lze pracovat. Souvisí s viditelností – platná proměnná může být skryta jinou stejného jména.

Python – pravidlo **LEGB**: **L**ocal (uvnitř funkce) → **E**nclosing (vnější funkce) → **G**lobal (modul) → **B**uilt-in (vestavěné).

2023

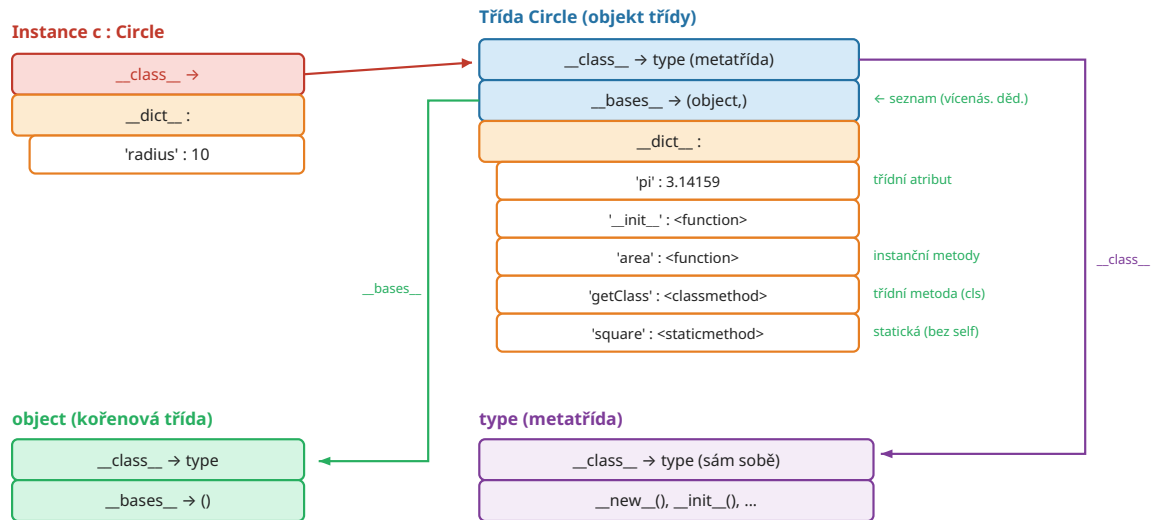
(A)

1) Sémantika třídní dědičnosti.

= sdílení položek nadřídou podtřídou + možnost přidání nových položek v podtřídě + možnost redefinice (překrytí) zděděných metod. Nelze popisovat slovem „dědí“ – to je definice kruhem!

2) Instance v Python 3 + skutečné podtypy vs vyžádaná dědičnost.

Třída a instance v Python 3



Lookup pravidla:

`c.radius` → `c.__dict__['radius']` (instancí atribut)

`c.pi` → `c.__class__.__dict__['pi']` (třídní atribut přes instanci, pokud nemá inst. atribut stejného jména)

Instance Python 3: `__class__` (odkaz na vlastní třídu) + `__dict__` (slovník hodnot instancí atributů). Neobsahuje metody – ty se dohledávají přes třídu.

- **Skutečné podtypy** (Python, Smalltalk): metoda se hledá **dle jména** ve slovníku metod. Nezávisí na dědičnosti – stačí mít potřebné metody (duck typing).
- **Vyžádaná dědičnost** (Java, C++): metoda se hledá přes **offset ve VMT** – rychlejší, ale vyžaduje dědičnost pro typovou kompatibilitu.

3) Rozhraní + výhoda oproti jednoduché dědičnosti – viz 2025 C1.

4) C struct – viz 2025 B4.

5) Doba života – viz 2024 C4.

(B)

1) Veřejný protokol – viz 2025 A1.

2) GC podmínky – viz 2025 F2.

3) Dynamická dědičnost, prototyp, rys – rozdíl.

- **Rys (trait)** = objekt obsahující **metody** (sdílené chování). Simuluje třídu – slouží k sdílení kódu. Nemá data.
- **Prototyp** = objekt obsahující **datové sloty** s implicitními hodnotami + `parent*` na rys. Šablona ke klonování – simuluje instanci. Nové objekty vznikají kopírováním prototypu.
- **Dynamická dědičnost** = hodnoty `parent*` slotů lze měnit za běhu → objekt dynamicky mění svého předka a tím i chování.

4) Union v C – viz 2024 E4.

5) Sémantika + C kontroly – viz 2024 A4.

(C)

1) Override vs overload.

- **Override (překrytí/redefinice):** nová implementace metody v podtřídě se stejnou signaturou jako v nadtřídě. Vyžaduje dědičnost.
- **Overload (přetížení):** více metod stejného jména ve třídě, liší se typem nebo počtem parametrů. Nepotřebuje dědičnost.

2) Explicitní rušení – viz 2024 C1.

3) Self aktivační objekt v SELF – viz 2025 C2.

4) Adresa $a[x]$, $\text{base}=24$: $\text{adresa}(a[x]) = 24 + (x - 13) \times 4$

5) Syntaxe + rys PHP – viz 2024 D4.

(D)

1) Abstrakce/objekt v OOP.

Objekt = entita zapouzdřující data (atributy) a operace (metody), autonomní, výpočetně úplná, s unikátní identitou nezávislou na hodnotách atributů. **Abstrakce** = schopnost zjednodušit/ignorovat aspekty objektu – chová se jako black box definovaný pouze rozhraním.

2) VMT obrázek – viz 2024 A2.

3) 3 nevýhody GC.

1. **Prostorová reže** – ukládání metadata (počty referencí)
2. **Časová reže** – udržování informací o referencích + samotné provádění GC
3. **Nedeterminismus** – GC se může spustit v nepředvídatelný/nehodný čas

4) 6 vlastností proměnné + PHP – viz 2025 C4.

5) Rozsah platnosti + Python – viz 2024 E5.

(E)

1) Třída a instance v Python 3.

Třída v Python 3:

- Třídní atributy (definované přímo v těle třídy, mimo metody)
- Definice instančních metod (vč. `__init__`), třídních metod (`@classmethod`, param `cls`), statických metod (`@staticmethod`, bez `self`)
- `__bases__` – **seznam** nadtříd (Python podporuje vícenásobnou dědičnost!)
- `__class__` → `type` (metatřída)
- `__dict__` – slovník pro hledání položek dle jména

Instance v Python 3:

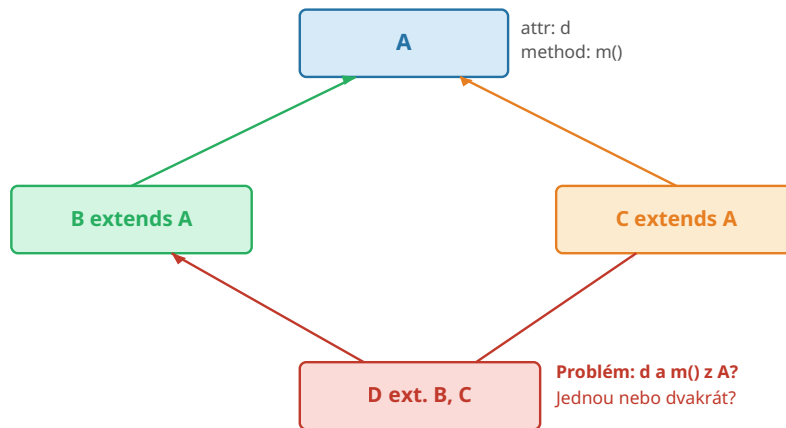
- `__class__` – odkaz na vlastní třídu

- `__dict__` – slovník s hodnotami instančních atributů
- BEZ metod (dohledávají se přes třídu), BEZ identifikátoru/jména instance

2) Implementace polymorfismu: staticky typ. vs skutečné podtypy – viz 2023 A2.

3) Diamantový problém.

Diamantový problém – vícenásobná dědičnost



Řešení (bez změny hierarchie):

- Sloučení – atribut d jednou, hledání přes jméno (Python 3 – MRO linearizace)
- Repeated inheritance – d dvakrát, mohou se lišit (C++ bez virtual)
- Virtual inheritance – d jednou (C++ s virtual)

D dědí od B a C, obě dědí od A → atributy/metody z A jednou nebo dvakrát? Řešení: **sloučení** (Python MRO – jednou), **repeated inheritance** (C++ bez virtual – dvakrát, mohou se lišit), **virtual inheritance** (C++ s virtual – jednou).

4) Abstrakce řízení toku.

- **Imperativní:** programátor specifikuje **co** a **jak** (v jakém pořadí). Příkazy, větvení, cykly. PŘ: C, Java, Python.
- **Deklarativní:** programátor specifikuje pouze **co**. Pořadí řeší systém, opakování = rekurze. PŘ: Haskell, Prolog, SQL.

5) Typovost jazyků + co určuje typ.

- **Beztypové** – typ nemá smysl, jazyky nepracují s hodnotami v proměnných (λ -kalkul, ζ -kalkul). NEMAJÍ jeden univerzální typ!
- **Netypované** – typ dán hodnotou, mění se za běhu, implicitní konverze (PHP, Python, BASIC)
- **Typované** – typ určen ve zdrojovém kódu, explicitně (anotací) nebo odvozen (typová inference). PŘ: Java, C++, Haskell.

Z typu proměnné: **množina hodnot**, **množina operací**, **velikost paměti**.

(F)

1) Identita objektu.

= porovnání, zda jde o **tentýž** objekt. Implementace: implicitně **adresou v paměti**, nebo unikátním ID. Identita $\neq$ shodnost – dva objekty mohou mít stejné hodnoty atributů ale různou identitu. Identita implikuje shodnost, ale ne naopak.

2) GC – viz 2025 F2.

3) Interface + náhrada v OOJ s vícenásobným děděním – viz 2025 C1.

4) Definice vs deklarace – viz 2025 A3.

5) Binární sčítání – viz 2024 D5.

2022

21.03 – Běh 1

1) Override vs overload + syntaktická konstrukce.

Override: nová implementace v podtřídě (stejná signatura). Syntaktická konstrukce: `virtual` u metody v nadtřídě (C++), `override` v podtřídě pro kontrolu (C#, C++11). **Overload:** více metod stejného jména, různé parametry, nepotřebuje dědičnost.

2) Kolize atributů + 4 řešení – viz 2024 D3.

3) Vícetypový objekt + 4 operace.

= objekt s více typy současně (pochází z OO databází, tzv. role). Objekt může za běhu dynamicky nabývat/pozbývat role. Operace: 1) přidání role 2) odebrání role 3) zjištění zda má roli 4) přístup k položkám přes konkrétní roli.

4) Syntaxe + PHP – viz 2024 D4.

5) Definice vs deklarace – viz 2025 A3.

21.03 – Běh 2

1) VMT – polymorfní vs nepolymorfní metoda.

VMT = tabulka ukazatelů na kód polymorfních metod. Každá třída má svou VMT, instance obsahuje odkaz na VMT.

- **Polymorfní (virtuální):** uloženy ve VMT → **pozdní vazba** (metoda určena za běhu dle skutečného typu)
- **Nepolymorfní:** NEJSOU ve VMT → **brzká vazba** (metoda určena při překladu, kód přímo)

2) Imperativní vs deklarativní – viz 2023 E4.

3) Rozhraní – viz 2025 C1.

4) Dědičnost – účel a význam.

Sdílení/znovupoužití položek (metod, atributů) skrze třídy + zajištění zpětné kompatibility rozhraní. Podtřída: sdílí položky nadtřídy, může přidávat nové, může redefinovat zděděné metody.

5) Doba života v PHP.

= interval alokace paměti. V PHP: **lokální** = od vstupu do funkce do opuštění, **globální** = celý běh skriptu, **statické lokální** (`static`) = celý skript ale viditelné jen ve funkci, **dynamické** = od vytvoření do `unset()`.

21.03 – Běh 3

1) Minimální model OO.

1. **Pojmenování objektu** (přiřazení do proměnné) 2. **Zaslání zprávy** objektu (selektor + parametry). Pro minimální model NEpotřebujeme: třídy, dědičnost, atributy (nahrazeny metodami), zapouzdření, parametry metod (stačí skrytý self).

2) Třída a instance v Python 3 – viz 2023 E1.

3) Kolize metod + 4 řešení – viz 2025 A2.

4) Sémantika + C – viz 2024 A4.

5) Netypovaný + * v Pythonu – viz 2024 C5.

22.03 – Běh 1

1) Zpráva + protokol + pozdní vazba.

Zpráva: selektor + příjemce + argumenty. Souvislost s pozdní vazbou: metoda se určuje **za běhu** dle skutečného typu příjemce → stejná zpráva může vyvolat různé metody u různých příjemců = polymorfismus.

2) Polymorfismus bez dědičnosti.

Lze realizovat: **rozhraní** (interface), **přetěžování** (overloading), **duck typing** (dynamicky typované OOJ – stačí mít potřebné metody).

3) SELF – viz 2023 B3.

4) Rozsah platnosti Python – viz 2024 E5.

5) Vlastnosti proměnné + PHP – viz 2025 C4.

22.03 – Běh 2

1) Veřejný protokol – viz 2025 A1.

2) Zánik instancí – viz 2024 C1 + 2025 F2.

3) SELF – objekty a sloty.

Objekt v SELF = seznam **slotů**: **datový** (hodnota/reference), **metodový** (kód metody), **rodičovský** (parent* → rodičovský objekt pro delegaci). Nerozlišuje se instance/třída.

4) Typovost – viz 2023 E5.

5) Otevřený podprogram.

Uložen v rámci hlavního zdrojového textu. **Nemá** definované rozhraní (parametry, návratovou hodnotu). Vstup skokem (GOSUB), výstup příkazem **RETURN** . Parametry i výsledky přes **globální proměnné** – nemá lokální proměnné. Nepodporuje rekurzi ani vnořování. Nebezpečí: přepsání cizích dat.

22.03 – Běh 3

1) Zahrnutí (subsumption).

Pokud B je podtřída A, lze instanci B využít kdekoli je očekávána instance A. Statický přístup k instanci B přes protokol A. Proto ve staticky typovaných OOJ nelze rušit položky.

2) Skutečné podtypy + 2 jazyky.

Kompatibilita dle skutečně implementovaných položek (ne dle dědičnosti) = **duck typing**. Polymorfismus nezávislý na třídní dědičnosti. Jazyky: **Python**, **Smalltalk** (příp. SELF, Objective-C).

3) Sloty klonovaného objektu v SELF.

Klonování = mělká kopie → nový objekt se stejnými sloty (data s implicitními hodnotami, parent* na stejný rys). Po klonování modifikace datových slotů. Při invokaci metody: self = klonovaný objekt (původní příjemce).

4) Sémantika – 4 druhy popisu.

Axiomatická (axiomy pro každou konstrukci), **operační** (přechody mezi stavy), **denotační** (program jako matematická funkce), **neformální** (slovní popis + příklady).

5) Binární sčítání – viz 2024 D5.

2019

(B)

1) Kolize metod – viz 2025 A2.

2) Netypované + * v Pythonu – viz 2024 C5.

3) Zpráva + veřejný protokol – viz 2025 A1.

4) Prototyp – viz 2023 B3.

5) Sémantika + C – viz 2024 A4.

(C)

1) Třída a instance Python 3 – viz 2023 E1.

2) B podtyp A – viz 2022 22.03 ot.1.

3) Definice vs deklarace – viz 2025 A3.

4) Delegace vs předání zprávy v BOOJ.

Předání zprávy (forwarding): self se **změní** na nového příjemce. **Delegace:** self **zůstává** původní příjemce – metoda v rodičovském objektu pracuje v kontextu původního příjemce.

5) Syntaxe – viz 2024 D4.

(?)

1) Zpráva + interní protokol.

Interní protokol (objekt zasílá zprávu sám sobě – self/this): a) přístup k atributu (čtení nebo modifikace) b) invokace metody → výsledek c) chyba DoesNotUnderstand.

2) Nevýhody GC – viz 2023 D3.

3) SELF sloty – viz 2022 22.03 Běh 2 ot.3.

4) Typovost – viz 2023 E5.

5) Otevřený podprogram – viz 2022 22.03 Běh 2 ot.5.

(D)

1) Rušení instancí – viz 2024 C1 + 2025 F2.

2) Abstraktní třída + srovnání – viz 2025 C1.

3) Vícetypový objekt – viz 2022 21.03 Běh 1 ot.3.

4) Vlastnosti proměnné + PHP – viz 2025 C4.

5) Rozsah platnosti + Python – viz 2024 E5.

2018

(A)

1) Objekt + 3 koncepty OOP.

Objekt = entita zapouzdřující data (atributy) + operace (metody), autonomní, výpočetně úplná, s unikátní identitou. **3 koncepty**: zapouzdření (přístup jen přes protokol), polymorfismus (stejná zpráva → různé reakce), dědičnost (sdílení a rozšiřování chování).

2) Redefinice metody + syntaktická konstrukce + proč umožněno.

Redefinice (overriding) = nová implementace v podtřídě se stejnou signaturou jako v nadtřídě. Pro přístup k původní metodě: `super` (Java, Python), `base` (C#), `parent::` (PHP), `A::m()` (C++).

Umožněno proto, aby šlo novou implementaci **postavit na původní** – pouze rozšířit existující kód místo kompletní reimplementace.

3) Vícetypový objekt – viz 2022 21.03 Běh 1 ot.3.

4) Rozsah platnosti + PHP.

Lokální (ve funkci), **globální** (mimo funkce – nutno `global` pro přístup z funkce), **statický lokální** (`static` – přežívá mezi voláními funkce), **superglobální** (`$_GET`, `$_POST`, `$_SESSION` ...).

5) Typované jazyky + situace při přiřazení.

Typované = typ určen ve zdrojovém kódu. Při přiřazení `a = b` : stejný typ → OK; podtyp → OK (zahrnutí); kompatibilní → implicitní konverze (koerce); nekompatibilní → typová chyba; explicitní přetypování → s rizikem ztráty dat.

(B)

1) Instanciace – viz 2024 E1.

2) Rozhraní – viz 2025 C1.

3) SELF – viz 2022 22.03 Běh 2 ot.3.

4) Sémantika – formální popisy – viz 2022 22.03 Běh 3 ot.4.

5) Vlastnosti proměnné + PHP – viz 2025 C4.

(C)

1) Zpráva + interní protokol – viz 2019 ? ot.1.

2) Zahrnutí typu – viz 2022 22.03 Běh 3 ot.1.

3) Kolize metod + 4 řešení – viz 2025 A2.

4) Typovost – viz 2023 E5.

5) Otevřený podprogram – viz 2022 22.03 Běh 2 ot.5.

(E)

1) Třída + metatřída.

Třída: seznam inst. atributů (metadata) + implementace inst. metod + třídní proměnné/metody + odkaz na nadtřidu. **Metatřída:** v čistém OOJ (Smalltalk) je třída také objekt → má svou třídu = metatřída. Metatřída definuje třídní zprávy (např. pro instanciaci). Smyčka: metatřída metatřídou = ona sama.

2) Rušení instancí – viz 2024 C1 + 2025 F2.

3) Vícetypové objekty – viz 2022 21.03 Běh 1 ot.3.

4) Syntaxe + Python – viz 2025 C3.

5) Definice a deklarace – viz 2025 A3.

(B)

1) Zpráva + veřejný protokol – viz 2025 A1.

2) Rušení instancí – viz 2024 C1 + 2025 F2.

3) Rys při simulaci třídního modelu v SELF.

Rys (trait) = objekt se sdílenými metodami, simuluje třídu. Objekty na něj odkazují přes parent* sloty a delegují – neznámé zprávy se hledají v rysu. Více objektů sdílí stejný rys = sdílení implementace jako u třídní dědičnosti.

4) Syntaxe – viz 2025 C3.

5) Uzavřený podprogram + rozdíl od otevřeného.

Uzavřený: MÁ rozhraní (jméno, parametry, návratový typ), lokální proměnné, vstup voláním (invokací), výstup návratem výsledku. Podporuje rekurzi a vnořování. **Otevřený:** NEMÁ rozhraní, parametry přes globální proměnné, vstup skokem, BEZ lokálních proměnných, BEZ rekurze.

(D)

1) Dědičnost – viz 2022 21.03 Běh 2 ot.4.

2) Polymorfní vs nepolymorfní – viz 2022 21.03 Běh 2 ot.1.

3) SELF – viz 2022 22.03 Běh 2 ot.3.

4) Typovost – viz 2023 E5.

5) Otevřený podprogram – viz 2022 22.03 Běh 2 ot.5.

(?)

1) Třída + metatřída – viz 2018 E1.

2) Rozhraní – viz 2025 C1.

3) Kolize při vícenásobné dědičnosti – viz 2025 A2.

4) Rozsah platnosti PHP – viz 2018 A4.

5) Imperativní vs deklarativní – viz 2023 E4.

(C)

1) Self/this parametr – viz 2025 B1.

2) Doba života + PHP – viz 2024 C4.

3) Delegation + self v BOOJ – viz 2025 B2 + 2019 C4.

4) Třída a instance čistě třídního OOJ.

Třída (5): inst. atributy (metadata), implementace inst. metod, třídní proměnné, třídní metody, odkaz na nadtřidu. **Instance (3):** identita (unikátní), odkaz na vlastní třídu, hodnoty inst. atributů.

5) Bitové pole – `word = *(A + 8); word = word >> 7; result = word & 0xF;`

(?)

1) Definice objektu – viz 2018 A1.

2) Vlastnosti proměnné + problémy ukazatelů.

6 vlastností: jméno, adresa, typ, hodnota, doba života, rozsah platnosti. Problémy ukazatelů: **dangling pointer** (ukazuje na uvolněnou paměť), **memory leak** (ztráta odkazu), **aliasing** (více ukazatelů na totéž), **NULL dereference**.

3) SELF – viz 2022 22.03 Běh 2 ot.3.

4) Floyd-Hoarova logika.

Formální systém pro důkaz korektnosti programů. Trojice **{P} S {Q}**: P = preconditione (co platí před), S = příkaz, Q = postconditione (co platí po). Vychází z axiomatické sémantiky a predikátové logiky. Pravidla pro přiřazení, sekvenci, if, while.

5) Rozhraní – viz 2025 C1.

(F)

1) VMT – účel + implementace pro jednoduchou dědičnost.

Účel: implementace pozdní vazby pro polymorfní metody. **Implementace:** každá třída má flatVMT s ukazateli na kód polym. metod (stejně offsety pro zděděné). Instance = Ref(VMT) + atributy. Podtřída kopíruje strukturu VMT nadtříd – redefinované = nové ukazatele. Invokace: instance → VMT → offset → kód.

- 2) Sémantika – viz 2022 22.03 Běh 3 ot.4.
- 3) Kolize metod – viz 2025 A2.
- 4) Rušení instancí – viz 2024 C1 + 2025 F2.
- 5) Netypovaný + binární sčítání – viz 2024 D5.

(D)

- 1) Dědičnost – viz 2022 21.03 Běh 2 ot.4.
- 2) Polymorfní vs nepolymorfní – viz 2022 21.03 Běh 2 ot.1.
- 3) Prototyp – viz 2023 B3.
- 4) Typovost – viz 2023 E5.
- 5) Otevřený podprogram – viz 2022 22.03 Běh 2 ot.5.

(E)

- 1) Zpráva + veřejný protokol – viz 2025 A1.
 - 2) 2 způsoby instanciace.
 - 1. Klíčové slovo **new** (hybridní OOJ) – `obj = new Class(args);`
 - 2. Třídní zpráva (čistý OOJ) – `obj := ClassName new.`
 - 3) Rys – viz 2017 B3.
 - 4) Rozsah platnosti PHP – viz 2018 A4.
 - 5) Typované jazyky – viz 2018 A5.
-

2015

(A)

- 1) Zpráva + veřejný protokol – viz 2025 A1.
- 2) Statická vs třídní metoda.

Statická: BEZ self-parametru – funkce ve jmenném prostoru třídy, nemůže přistupovat k inst. ani třídním proměnným přes self. **Třídní:** self = objekt třídy – může přistupovat k třídním proměnným. Obě nemohou přistupovat k instančním atributům. V hybridních OOJ (Java, C++) existují jen statické, třídní metody nemají.

3) Sémantika – viz 2022 22.03 Běh 3 ot.4.

4) Beztypové jazyky – viz 2023 E5 (beztypové).

5) Vyžádaná dědičnost vs skutečný typ – viz 2023 A2.

(?)

1) Definice objektu – viz 2018 A1.

2) Pozdní vazba + kód invokace.

Pozdní vazba = metoda určena za běhu dle skutečného typu příjemce. Kód invokace `o.m()` :

`Load Reg, [addr(o)] → Add Reg, #offset_VMT → Load Reg, [Reg] → Add Reg, #offset_m → Call [Reg]` . **Brzká vazba** = metoda určena při překladu, překladač přímo vloží adresu kódu.

3) SELF – viz 2022 22.03 Běh 2 ot.3.

4) Typovost – viz 2023 E5.

5) Otevřený podprogram – viz 2022 22.03 Běh 2 ot.5.

(?)

1) Zpráva + interní protokol – viz 2019 ? ot.1.

2) Třídní vs instanční metoda.

Instanční: `self` = instance → přístup k inst. atributům. **Třídní**: `self` = třída → BEZ přístupu k inst. atributům (instance nemusí existovat).

3) Rozhraní – viz 2025 C1.

4) Rozsah platnosti C/C++/Java.

Lokální (blok `{ }`), funkční (parametry), třídní (atributy třídy – Java/C++), globální (mimo funkce – C/C++), jmenný prostor (namespace/package).

5) Typované jazyky + přiřazení – viz 2018 A5.

(E)

1) Syntaxe + 2 hlášky ne-syntaktické.

Syntaxe = definice struktury programu. Popis: (E)BNF, syntaktické grafy, BK gramatiky. 2 hlášky překladače, co NEJSOU syntaktické (= sémantické): „proměnná x nebyla deklarována“, „nekompatibilní typy v přiřazení“.

- 2) Uzavřený podprogram – viz 2017 B5.
- 3) Rysy – viz 2017 B3.
- 4) Minimální model – viz 2022 21.03 Běh 3 ot.1.
- 5) Rušení instancí – viz 2024 C1 + 2025 F2.

(?)

1) Druhy dědičnosti.

Dle počtu předků: **jednoduchá** (max 1 přímý předek) / **vícenásobná** (více). Dle předmětu: **dědičnost implementace** (metody včetně kódu + atributy) / **dědičnost rozhraní** (jen deklarace metod – protokol).

2) Instanciace – viz 2024 E1.

3) Prototypy – viz 2023 B3.

4) Vlastnosti proměnné + ukazatele – viz 2016 ? ot.2.

5) Floyd-Hoarova logika – viz 2016 ? ot.4.

Samostatné otázky

6) Diamant v C++.

Problém: duplicitní atributy/metody z kořenové třídy, nejednoznačnost. Řešení: `virtual` u obou nadtříd při dědění → virtual inheritance (atribut jen jednou).

7) Co z typu proměnné.

Velikost paměti, obor hodnot, množina operací.

8) Dynamická vícenásobná dědičnost v SELF.

Více **parent*** slotů (vícenásobná) + měnitelné za běhu (dynamická) = delegace.

9) Přístup k datům v minimálním modelu (bez atributů).

Atributy nahrazeny **metodami** – čtení = invokace metody, modifikace = modifikace kódu metody (operace ⇐ v ζ -kalkulu). Vše přes zasílání zpráv.

10) Statické metody.

BEZ self-parametru → nemohou přistupovat k položkám instance. Definovány uvnitř třídy s klíčovým slovem `static`. Jako funkce ve jmenném prostoru třídy.

11) Parametry u otevřených podprogramů.

Přes **globální proměnné** (neexistují lokální). Nebezpečí: přepsání cizích dat, nemožnost rekurze, složitá údržba.

12) Definice rozhraní v Javě.

Deklarace metod (implicitně public) + nadrozhraní (`extends`) + případně konstanty. Od Java 8 i default metody.

13) `super.m()` – brzká nebo pozdní vazba?

Brzká (statická) – při překladu je jednoznačně známo, ve které nadtřídě se má hledání začít.

14) Více proměnných na jedné adrese?

Ano. Pointery v C (aliasing), reference v C++, předávání odkazem, union.

15) Vícetypový objekt (role) vs vícenásobná dědičnost.

Dědičnost: typy určeny **staticky** při definici třídy. Role: typy se mění **dynamicky** za běhu života objektu.

16) Destruktor vs finalizační metoda.

GC (implicitní): Java, Python. Explicitní: C++. **Destruktor**: volaný programátorem/delete, garantované vykonání, předvídatelný čas. **Finalizace**: volaná GC, vykonání NENÍ garantováno, čas nepředvídatelný.

17) Třídní atribut Python 3.

Definuje se **přímo v těle třídy** (mimo metody): `pi = 3.14`. Přístup přes instanci možný, pokud instance **nemá** instanční atribut stejného jména (Python hledá nejprve v `instance.__dict__`, pak v `class.__dict__`).

18) flatVMT(B) pro A::m nepolymorfní, A::k polymorfní, B redefinuje obě.

$\text{flatVMT}(B) = \{ B::k \}$. Nepolymorfní m se do VMT NEUKLÁDÁ – řeší se staticky.